OSTEP Chapter 1 - Introduction to Operating Systems

Goal of an OS: provide a virtualized, safe, and convenient environment for running programs

Main roles of an OS:
provide a virtualized, safe and convenient environment for running programs.

Main roles of an OS
1. Abstraction
→ Hides hardware complexity from programs
→ provides clean interfaces (files, process, virtual memory)

2. Resource Management
→ CPU Scheduling
→ Memory allocation
→ I/O control
→ Fairness + efficiency

3. Control / protection
→ Isolates processes
→ prevents interference
→ Enforces permissions

Key Idea: Virtualization
→ OS makes one physical machine look like many independent logical machines

Example:
Virtual CPU → process abstraction
virtual memory → address space
virtual disks → file system

Date No

Chapter 2 → Processes (The Abstraction)

This chapter defines what a "process" means in OSTEP, why it's important and how OS virtualizes the CPU.

→ A process is a running program. The static program on disk becomes a "process" once loaded & executed.

→ A process's state comprises:
  → Its address space (memory: code, stack, heap, static data)
  → CPU-related state: registers, program counter (PC), stack pointer — everything needed to resume execution later
  → OS-tracked metadata: open files / I/O state, process ID (PID), scheduling information, etc

→ Because there are limited physical CPUs, the OS uses CPU virtualization / time-sharing switching between processes so that it "appears" multiple processes run simultaneously.

→ Processes can be in multiple states: commonly Running, Ready (ready to run, waiting to be scheduled), or Blocked (waiting for I/O or other event)

This abstraction is powerful: it lets users run many programs concurrently - even hundreds - even with one (or few) CPUs underlying

Chapter 3 → Process API (creation, control, lifecycle)

This chapter discusses how an OS allows programs to create & manage processes - what interfaces the OS gives to user-programs

→ In UNIX-style systems, typical system calls for processes include:
- fork() - creates a new process (child) as a (nearly) identical copy of the parent
- exec() - replace the process's memory image with a new program (after fork, often) so that the child can start executing a different program
- wait() - parent waits for child processes to finish; helps manage child lifetime & prevent zombie processes

FALCON

Date No

- exit() – terminate a process. OS cleans up process resources; parent may check exit status

→ The book shows a typical workflow: a program calls fork(), child does exec() to load a new program, parent does wait()

→ It also touches upon the "process control block" (PCB) – the internal data structure OS uses to track all necessary info about each process (state, registers, memory, open files, etc.) so it can suspend/resume processes (i.e. context switching.)